

Documentation technique - Data Lake Finance

Installation, architecture, pipeline, endpoints et preuves de fonctionnement

Groupe

Artemiy Smogunov - Nathan Smadja-Tubiana - Patrice Ignongui

Projet Data Lake Finance - EFREI

Lien GitHub : <https://github.com/Nathan2412/Projet-DataLake-Final>

Version générée le 09/07/2026 08:19

Objectif : construire un data lake financier complet, depuis l'ingestion de données brutes jusqu'à l'analyse curated, avec orchestration Airflow, stockage MinIO, indexation Elasticsearch, base PostgreSQL, API FastAPI et contrôles de qualité.

1. Lancement du projet

Dépôt GitHub : <https://github.com/Nathan2412/Projet-DataLake-Final>

Le projet se lance avec Docker Compose. Les services principaux exposés sont : API FastAPI sur le port 8000, Airflow sur 8080, MinIO sur 9001, PostgreSQL sur 5432, Elasticsearch sur 9200 et Redis sur 6379.

Service	URL / port	Rôle
FastAPI	http://localhost:8000/docs	Démo, ingestion, statistiques, santé
Airflow	http://localhost:8080	Orchestration du DAG financial_data_lake_pipeline
MinIO	http://localhost:9001	Stockage objet raw
Elasticsearch	http://localhost:9200	Indexation raw consultable
PostgreSQL	localhost:5432	Staging, curated et logs
Redis	localhost:6379	Cache et accélération API

Commandes de référence

```
docker compose up -d
docker compose ps
docker exec projet-data_lake_finace-api-1 python -m unittest discover -s /app/tests -v
```

2. Pipeline technique

Étape	Entrée	Sortie	Contrôle
Ingestion	Tickers yfinance / fichiers	Objets MinIO + docs Elasticsearch	Logs d'ingestion et statut endpoint
Staging	Docs raw par ticker	Tables nettoyées OHLCV + indicateurs	Déduplication date, types numériques, close non nul
Curated	Staging	Scores anomalie, signal, tendance	IsolationForest, seuils métier, upsert
Orchestration	Planning Airflow	DAG planifié ou manuel	Historique des runs, santé scheduler
Exposition	Base + index	API /stats, /health, /docs	Swagger et tests unitaires

Le code de staging calcule notamment SMA20/50, EMA12/26, RSI14, MACD, bandes de Bollinger, daily_return et volatility_20. Le code curated réutilise ces variables, ajoute volume_zscore, applique IsolationForest puis classe les anomalies en catégories comme price_spike, flash_crash, volume_spike ou high_volatility.

3. État vérifié des services

Service	Statut	Détail
postgresql	ok	-
minio	ok	Buckets : ['raw-api-data', 'raw-financial-data']
elasticsearch	ok	version 8.11.0
redis	ok	-

Capture API /health - services du Data Lake

Reponse locale verifiee, pas une maquette

```
{
  "overall": "ok",
  "services": {
    "postgresql": {
      "status": "ok",
      "details": null
    },
    "minio": {
      "status": "ok",
      "details": "Buckets : ['raw-api-data', 'raw-financial-data']"
    },
    "elasticsearch": {
      "status": "ok",
      "details": "version 8.11.0"
    },
    "redis": {
      "status": "ok",
      "details": null
    }
  }
}
```

Repository: [https://github.com/Nathan2412/Projet-DataLake-Final](#)

Capture construite à partir de la réponse réelle /health.

Airflow indique scheduler et base de métadonnées en état healthy. Les champs dag_processor et triggerer peuvent être nuls dans cette configuration locale ; cela n'empêche pas le DAG de fonctionner car le scheduler et le webserver sont bien actifs.

4. Résultats techniques et justification

563 Objets raw MinIO	5958 Docs Elasticsearch	5917 Lignes staging
4909 Lignes curated	208 Anomalies	4.24% Taux anomalie

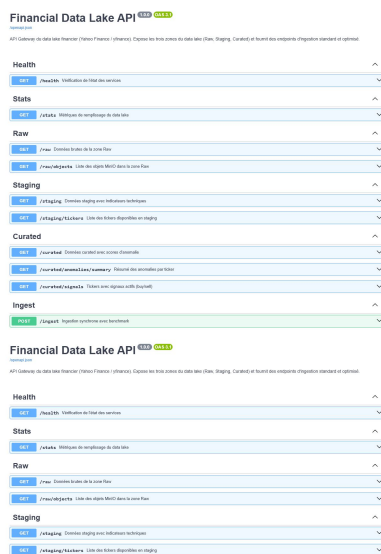
Question	Réponse technique
Pourquoi MinIO a moins de compteurs que Elasticsearch ?	MinIO compte des objets bruts. Elasticsearch compte les lignes extraites des fichiers, donc une seule archive peut produire de nombreux documents.
Pourquoi staging est inférieur au raw indexé ?	Le staging est la première couche de qualité : doublons date/ticker, valeurs non numériques ou close manquants sont corrigés ou écartés.
Pourquoi curated est inférieur à staging ?	4 tickers (1 008 lignes) sont encore présents seulement en staging. Ce point est identifié et actionnable.
Pourquoi le taux d'anomalies est 4,24% ?	IsolationForest cible 5%, mais les tickers très courts sont conservés sans apprentissage pour éviter des anomalies statistiquement faibles.

Ticker	Lignes	Anomalies	Taux	Dernière date
AAPL	868	44	5.1 %	2026-07-08
AMZN	263	14	5.3 %	2026-07-08
BRK-B	263	14	5.3 %	2026-07-08
GOOGL	263	14	5.3 %	2026-07-08
JNJ	263	14	5.3 %	2026-07-08
JPM	263	14	5.3 %	2026-07-08
META	263	14	5.3 %	2026-07-08
NVDA	263	14	5.3 %	2026-07-08

5. Performance et endpoint avancé

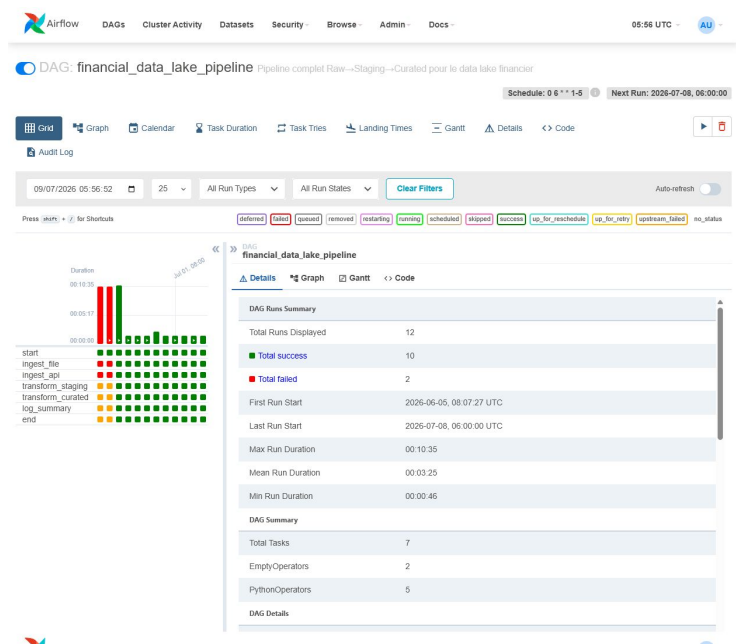
Lot	/ingest	/ingest_fast	Gain
1	4.16 s	1.86 s	55.20 %
100	70.63 s	23.61 s	66.57 %

La partie avancée est portée par /ingest_fast et par la couche curated. /ingest_fast combine parallélisme, upload MinIO parallèle, bulk Elasticsearch, vectorisation NumPy et insertions PostgreSQL groupées. La couche curated ajoute un modèle non supervisé de détection d'anomalies et des signaux exploitables.

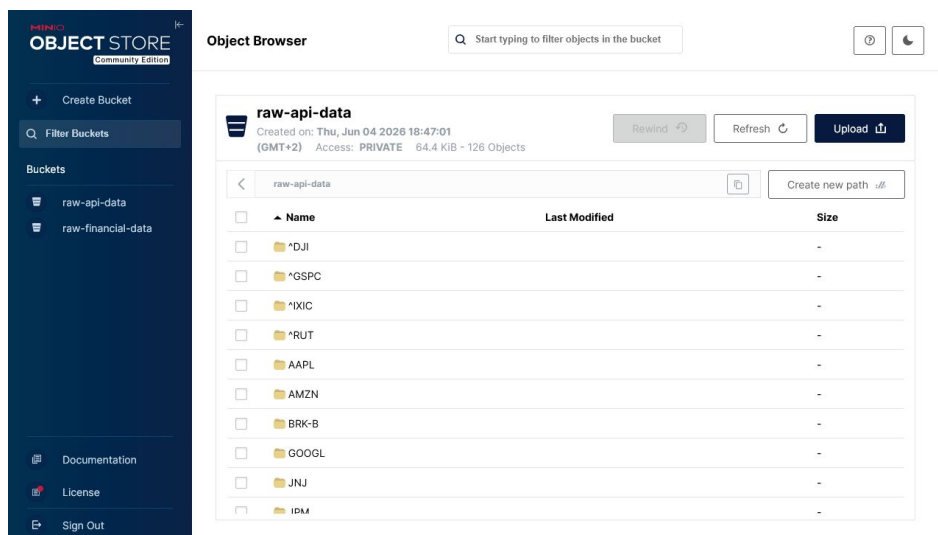


Swagger prouve les endpoints disponibles pour la démonstration.

6. Captures d'exploitation



Airflow : suivi du DAG, runs et tâches récentes.



MinIO : buckets raw utilisés par le data lake.

7. Vérification et limites

- Tests unitaires API exécutés dans le conteneur : 6 tests OK lors de la vérification finale.
- DAG Airflow actif avec derniers runs succès ; les échecs visibles en historique datent d'essais précédents.
- Les identifiants de démonstration sont volontairement simples pour le rendu local : Airflow admin/admin et MinIO minioadmin/minioadmin.
- La limite principale est la propagation des 4 indices internationaux en curated ; elle est isolée et documentée.
- Le dépôt contient un README et les consignes afin que l'enseignant puisse relancer le projet sans dépendre de cette conversation.

Cette documentation complète le rapport : elle donne les commandes, les ports, la logique du pipeline et les preuves de fonctionnement. Elle permet à un lecteur externe de comprendre non seulement comment lancer le projet, mais aussi pourquoi les résultats observés sont cohérents.